Продвинутые техники CSS

Мощные инструменты для создания сложной верстки без JavaScript

Докладчики: Барабанов Илья, Волков Иван

CSS-счётчики: автоматическая нумерация

CSS-счётчики позволяют управлять нумерацией элементов автоматически, без использования JavaScript. Это особенно полезно для создания оглавлений, нумерованных списков и сложной документации.

counter-reset

Создает или сбрасывает счётчик. Принимает имя счётчика и начальное значение (по умолчанию 0).

counter-increment

Увеличивает значение счётчика. Принимает имя счётчика и шаг (по умолчанию 1).

counter()

Вывод значения через свойство content в псевдоэлементах ::before или ::after.

Вложенные счётчики: иерархия

Как это работает

Вложенные счётчики позволяют создавать сложную иерархическую нумерацию, подобную той, что используется в .документации: **1.1, 1.2, 1.2.1**

Используется функция `counters(имя_счётчика, "разделитель")` вместо обычного counter(). Она автоматически собирает значения всех вложенных счётчиков с указанным именем.

Пример кода

```
section {
 counter-reset: subsection;
}
section h3::before {
 counter-increment:
subsection;
 content:
counters(subsection, ".") "
";
}
```

Пример

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* 1. counter-reset - создаём счётчик */
  .list {
    counter-reset: myCounter;
    list-style: none;
  }

  /* 2. counter-increment - увеличиваем счётчик */
  .list li {
    counter-increment: myCounter;
    margin-bottom: 10px;
  }

  /* 3. counter() - выводим значение */
  .list li::before {
    content: counter(myCounter) ". ";
    font-weight: bold;
    color: red;
  }
</style>
</head>
<body>
  <ul class="list">
    <li>Первый пункт</li>
    <li>Второй пункт</li>
    <li>Третий пункт</li>
  </ul>
</body>
</html>
```

Результат

1. Первый пункт
2. Второй пункт
3. Третий пункт

Многоколоночный текст

Разбивка блочного контента на несколько колонок для создания газетной верстки.

1 column-count

Задает количество колонок. Принимает целые числа или auto.

2 column-gap

Расстояние между колонками. Единицы измерения: px, em, % или normal.

3 columns

Короткая запись, объединяющая column-width и column-count (например, columns: 200px 3).

Пример

```
<!DOCTYPE html>
<html>
<head>
<style>
 .newspaper {
   column-count: 3;
   column-gap: 40px;
   column-rule: 2px solid #ccc;  /* линия между колонками */
   background: white;
   padding: 20px;
   font-family: Georgia, serif;
 }

 .newspaper h2 {
   column-span: all;  /* заголовок через все колонки */
   text-align: center;
   color: #c0392b;
 }
</style>
</head>
<body>
 <div class="newspaper">
   <h2>Газетный стиль</h2>
   <p>Этот текст оформлен как в газете — в три колонки с разделительной линией. column-count задаёт количество колонок, column-gap — расстояние между ними, а column-rule добавляет декоративную линию.</p>
   <p>Текст автоматически перетекает из колонки в колонку. Это удобно для создания журнальных или газетных макетов без использования сложных CSS Grid или Flexbox.</p>
   <p>Свойство column-span: all позволяет заголовку растянуться на все колонки. А column-gap контролирует отступы между колонками для лучшей читаемости.</p>
 </div>
</body>
</html>
```

Результат

Газетный стиль

Этот текст оформлен как в газете — в три колонки с разделительной линией. column-count задаёт количество колонок, column-gap — расстояние между ними, а column-rule добавляет декоративную линию.

Текст автоматически перетекает из колонки в колонку. Это удобно для создания журнальных или газетных макетов без использования сложных CSS Grid или Flexbox.

Свойство column-span: all позволяет заголовку растянуться на все колонки. А column-gap контролирует отступы между колонками для лучшей читаемости.

Клиппинг: обрезание элементов

Свойство **clip-path** скрывает часть элемента, оставляя видимой только определенную область или фигуру. Это мощный инструмент для создания нестандартных форм без использования изображений.

circle()

Круглая область обрезания

ellipse()

Эллиптическая область

polygon()

Многоугольная область

inset()

Внутреннее обрезание

path()

Сложные SVG-контуры

url()

Ссылка на SVG-маску

Пример

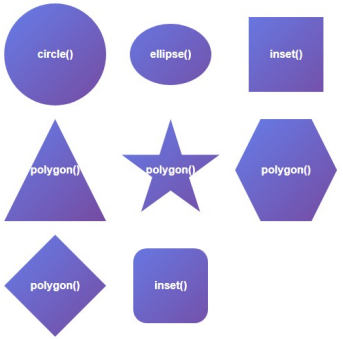
```html
<!DOCTYPE html>
<html>
<head>
<style>
 .container {
   display: flex;
   flex-wrap: wrap;
   gap: 20px;
   padding: 20px;
   font-family: Arial;
 }

 .box {
   width: 150px;
   height: 150px;
   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
   transition: all 0.3s;
   text-align: center;
   line-height: 50px;
   color: white;
   font-weight: bold;
 }

 .box:hover {
   transform: scale(1.05);
 }

 .circle { clip-path: circle(50%); }
 .ellipse { clip-path: ellipse(40% 30%); }
 .inset { clip-path: inset(20px); }
 .triangle { clip-path: polygon(50% 0%, 0% 100%, 100% 100%); }
 .star { clip-path: polygon(50% 0%, 61% 35%, 98% 35%, 68% 57%, 79% 91%, 50% 70%, 21% 91%, 32% 57%, 2% 35%, 39%
35%); }
 .hexagon { clip-path: polygon(25% 0%, 75% 0%, 100% 50%, 75% 100%, 25% 100%, 0% 50%); }
 .diamond { clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%); }
 .rounded-inset { clip-path: inset(20px round 20px); }
</style>
</head>
<body>
 <div class="container">
   <div class="box circle">circle()</div>
   <div class="box ellipse">ellipse()</div>
   <div class="box inset">inset()</div>
   <div class="box triangle">polygon()<br>треугольник</div>
   <div class="box star">polygon()<br>звезда</div>
   <div class="box hexagon">polygon()<br>шестиугольник</div>
   <div class="box diamond">polygon()<br>ромб</div>
   <div class="box rounded-inset">inset()<br>с round</div>
 </div>
</body>
</html>
```

Результат



CSS фигуры: полная таблица сравнения

Функция	Описание	Синтаксис	Пример
`circle()`	Круг	`circle(радиус at x y)`	`circle(50%)` `circle(100px at 30% 40%)`
`ellipse()`	Эллипс	`ellipse(радиусX радиусY at x y)`	`ellipse(40% 30%)` `ellipse(80px 50px at 50% 50%)`
`inset()`	Прямоугольник с отступами	`inset(верх право низ лево round радиус)`	`inset(20px)` `inset(10px 20px 30px 40px round 15px)`
`polygon()`	Многоугольник	`polygon(x1 y1, x2 y2, x3 y3, ...)`	`polygon(50% 0%, 0% 100%, 100% 100%)` `polygon(0% 0%, 100% 0%, 50% 100%)`
`path()`	SVG путь	`path("команды_пути")`	`path("M 100,30 C 130,0 170,0 200,30 Z")` `path("M 0,0 L 100,0 L 50,100 Z")`
`url()`	Ссылка на SVG маску	`url(#id_маски)`	`url(#heartClip)` `url(clipPath.svg#myClip)`

Маскирование: творческое обрезание

Как работает **mask**

Свойство **mask** использует изображение (обычно с прозрачным фоном или градиент) для скрытия части элемента. В отличие от clip-path, где видимость определяется формой, здесь видимость зависит от прозрачности маски.

Там, где маска непрозрачна — элемент виден. Там, где маска прозрачна — элемент скрыт.

Значения **mask**

- url() — путь к изображению
- linear-gradient() — линейный градиент
- radial-gradient() — радиальный градиент
- none — отсутствие маски

Мощность: mask — это шорткат, объединяющий mask-image, mask-mode, mask-repeat, mask-position и другие свойства.

Кастомизация маркера списка

Полный контроль над внешним видом "буллитов" в списках `<ul>` и `<ol>`.

::marker

Псевдоэлемент для прямой стилизации маркера (цвет, шрифт, размер)

list-style-type

Тип маркера: disc, circle, square, decimal и другие

list-style-image

Пользовательское изображение вместо стандартного маркера

list-style — короткая запись, объединяющая list-style-type, list-style-position и list-style-image

Практическое применение

Документация

Автоматическая нумерация разделов и подразделов с помощью счётчиков

Публикации

Многоколоночная верстка для газетных и журнальных макетов

Современный дизайн

Необычные формы через clip-path и маски для визуального выделения

UI-компоненты

Стилизованные списки для интерфейсов и пользовательских форм

Сравнение свойств

clip-path

- Создает геометрические формы
- Поддержка базовых фигур
- SVG-контуры через path()
- Анимация не всегда поддерживается

mask

- Использует изображения и градиенты
- Гибкость в дизайне
- Анимация через градиенты
- Требует прозрачности в изображении

Благодарим за внимание!

Ключевые навыки

Управление нумерацией, многоколоночная верстка, создание нестандартных форм

Следующий шаг

Практика применения этих техник в реальных проектах

Документация

MDN Web Docs — полное описание всех CSS-свойств

Докладчики: Барабанов Илья, Волков Иван

Блок 4. Практика: Задания для самостоятельной работы

Согласно требованиям, здесь мы готовим заготовки и описываем задачи на 5–15 минут. Ответы к ним в репозиторий добавлять не нужно, но вы должны уметь их решить «без шпаргалок».

Задание №1: "Фигурные изображения"

Задача: Используя свойство clip-path, превратите обычное квадратное изображение в восьмиугольник (октагон).

Заготовка: Создайте файл task1.html с тегом <img>.

Требование: Использовать функцию polygon().

Время: 10 минут.

Задание №2: "Газетная колонка"

Задача: Реализовать адаптивную раскладку текста. Текст должен автоматически разбиваться на колонки шириной не менее 250px, а между колонками должна появиться разделительная линия.

Заготовка: Файл task2.html с длинным текстом в теге <article>.

Требование: Использовать короткую запись columns и свойство column-rule.

Время: 5 минут.

Задание №3: "Автоматическое оглавление"

Задача: Настроить стили так, чтобы перед каждым заголовком <h2> автоматически выводился номер раздела в формате "Раздел №".

Заготовка: Файл task3.html с несколькими заголовками <h2>.

Требование: Использовать counter-reset, counter-increment и псевдоэлемент ::before.

Время: 15 минут.